

```
#!/usr/bin/env python3
```

```
"""
```

```
firewall.py – the Claim Firewall.
```

Sits between every generative agent and the user. Its single job:

NO SENTENCE REACHES A USER UNLESS EVERY FACTUAL CLAIM IN IT TRACES TO A FACT
THE FROZEN ENGINE ALREADY PROVED.

It fails CLOSED. If a claim cannot be verified, the sentence is dropped. If every
flourish sentence drops, the system falls back to the deterministic headline – wh
was never generated by an agent and therefore never needs firewalling.

Two layers, because fabrication has two shapes:

LAYER 1 – DETERMINISTIC SURFACE (this file, fully decidable, no LLM)

Catches invented ATOMS: numbers, years, dates, Saros numbers, eclipse types t
do not appear in the facts payload. A screenshot-able number that the engine
produced cannot pass. Years/dates are matched EXACTLY (2078 is not "close eno
to 2090). Physical quantities allow documented rounding (212 km -> "about 210

LAYER 2 – INDEPENDENT SEMANTIC (pluggable interface below, an LLM in production

Catches invented RELATIONS: sentences whose atoms are all real but recombined
a false claim ("the path passed directly over your birthplace" when inside_pa
False). The deterministic layer cannot see this; a separate verifier must.

The maker/checker rule holds: the semantic verifier MUST be a different agent, in
own context, from whatever narration agent produced the text. A checker that shar
maker's context is not a checker.

```
"""
```

```
from __future__ import annotations
```

```
import re
```

```
from dataclasses import dataclass, field
```

```
from typing import Callable, Optional
```

```
# ----- tolerances
```

```
YEAR_BAND = (1500, 2200)      # integers here are treated as years -> EXACT match
```

```
QUANTITY_REL_TOL = 0.02      # non-year quantities: within 2% of a real value, o
```

```
ROUNDING_SCALES = (-1, -2, -3) # ...a clean rounding of a real value (210 for 21
```

```
MONTHS = {m.lower(): i for i, m in enumerate(
```

```
    ["January", "February", "March", "April", "May", "June", "July",  
    "August", "September", "October", "November", "December"], 1)}
```

```
# Capitalized words that are not entities – never treated as place-name claims.
```

```
STOPWORDS = {
    "the", "your", "you", "a", "an", "in", "on", "at", "of", "and", "but", "or",
    "when", "where", "that", "this", "it", "its", "was", "will", "were", "shadow"
    "moon", "sun", "totality", "eclipse", "path", "born", "next", "last", "time",
    "years", "year", "day", "days", "over", "near", "just", "about", "only", "no"
    "saros", "series", "total", "annular", "hybrid", "partial", *MONTHS.keys(),
}
```

```
@dataclass
```

```
class Finding:
```

```
    layer: str          # "surface" | "semantic"
    kind: str           # "number" | "year" | "date" | "saros" | "eclipse_type" |
    token: str
    reason: str
```

```
@dataclass
```

```
class SentenceVerdict:
```

```
    sentence: str
    ok: bool
    findings: list = field(default_factory=list)
    provenance: list = field(default_factory=list) # payload atoms this sentence
```

```
@dataclass
```

```
class FirewallResult:
```

```
    passed: bool          # True only if the emitted flourish is fully cle
    emitted_flourish: str  # the sentences that survived (may be "")
    fell_back_to_headline: bool # True if nothing survived
    verdicts: list = field(default_factory=list)
```

```
    @property
```

```
    def blocked(self):
```

```
        return [f for v in self.verdicts for f in v.findings]
```

```
# ----- payload atom extract
```

```
def _walk(obj, key=None):
```

```
    """Yield (key, value) leaves from a nested facts payload."""
```

```
    if isinstance(obj, dict):
```

```
        for k, v in obj.items():
            yield from _walk(v, k)
```

```
    elif isinstance(obj, (list, tuple)):
```

```
        for v in obj:
```

```

        yield from _walk(v, key)
    else:
        yield (key, obj)

def build_allowlist(facts: dict) -> dict:
    """Derive the set of atoms an agent is permitted to state."""
    numbers, years, dates, saros, types, strings = set(), set(), set(), set(), se
    for key, val in _walk(facts):
        if isinstance(val, bool):
            continue
        if isinstance(val, (int, float)):
            numbers.add(float(val))
            if YEAR_BAND[0] <= val <= YEAR_BAND[1] and float(val).is_integer():
                years.add(int(val))
            if key and "saros" in key.lower():
                saros.add(int(val))
        elif isinstance(val, str):
            strings.add(val.lower())
            d = _parse_date(val)
            if d:
                dates.add(d)
                years.add(d[0])
                numbers.add(float(d[0]))
            if key and "type" in key.lower():
                types.add(val.lower())
            # allow every whitespace-token of a place/region string (e.g. "London
            for tok in re.findall(r"[A-Za-z][A-Za-z'-]+" , val):
                strings.add(tok.lower())
    return {"numbers": numbers, "years": years, "dates": dates,
            "saros": saros, "types": types, "strings": strings}

def _parse_date(s: str):
    s = s.strip()
    m = re.match(r"^(\d{3,4})-(\d{1,2})-(\d{1,2})$", s)          # ISO 1715-05-03
    if m:
        return (int(m.group(1)), int(m.group(2)), int(m.group(3)))
    m = re.match(r"^([A-Za-z]+\s+(\d{1,2})),?\s+(\d{3,4})$", s)  # August 12, 202
    if m and m.group(1).lower() in MONTHS:
        return (int(m.group(3)), MONTHS[m.group(1).lower()], int(m.group(2)))
    return None

# ----- number/date support
def _year_supported(x: int, allow) -> bool:
    return x in allow["years"]          # EXACT ONLY - years are unforg

```

```

def _quantity_supported(x: float, allow) -> bool:
    for n in allow["numbers"]:
        if x == n:
            return True
        if n != 0 and abs(x - n) / abs(n) <= QUANTITY_REL_TOL:
            return True
    for k in ROUNDING_SCALES:
        if round(n, k) == x:
            return True
    return False

# ----- LAYER 1: surface s
def surface_scan(sentence: str, allow: dict) -> SentenceVerdict:
    findings, prov = [], []

    # explicit dates first (so their years aren't re-judged as bare integers)
    consumed = set()
    for m in re.finditer(r"[A-Za-z]+\s+\d{1,2},?\s+\d{3,4}", sentence):
        d = _parse_date(m.group(0))
        if d and d in allow["dates"]:
            prov.append(f"date:{d[0]:04d}-{d[1]:02d}-{d[2]:02d}")
            consumed.update(range(m.start(), m.end()))
        elif d:
            findings.append(Finding("surface", "date", m.group(0),
                                    "date not present in facts payload"))
            consumed.update(range(m.start(), m.end()))

    # Saros references
    for m in re.finditer(r"[Ss]aros\s+(\d+)", sentence):
        n = int(m.group(1))
        consumed.update(range(m.start(), m.end()))
        if n in allow["saros"]:
            prov.append(f"saros:{n}")
        else:
            findings.append(Finding("surface", "saros", m.group(0),
                                    f"Saros {n} not in facts (payload has {sorted

    # eclipse types – a FIXED vocabulary. Any type word used as a claim must be o
    # payload actually asserts. If the payload names no type, none may be claimed
    # ('total' is exempt only as an adjective inside the stopworded phrase 'total
    # when facts assert a total; we key strictly off the payload's asserted type
    for t in ("annular", "hybrid", "partial", "total"):
        if re.search(rf"\b{t}\b", sentence, re.I) and t not in allow["types"]:
            findings.append(Finding("surface", "eclipse_type", t,

```

```

f"type '{t}' not asserted by facts (payload t

# bare numbers (skip anything already consumed by a date/saros span)
for m in re.finditer(r"\d[\d,]*\.\d*", sentence):
    if any(i in consumed for i in range(m.start(), m.end())):
        continue
    raw = m.group(0).replace(",", "")
    try:
        x = float(raw)
    except ValueError:
        continue
    is_year = x.is_integer() and YEAR_BAND[0] <= x <= YEAR_BAND[1]
    if is_year:
        if _year_supported(int(x), allow):
            prov.append(f"year:{int(x)}")
        else:
            findings.append(Finding("surface", "year", raw,
                                    f"year {int(x)} not in facts (exact match
    else:
        if _quantity_supported(x, allow):
            prov.append(f"number:{raw}")
        else:
            findings.append(Finding("surface", "number", raw,
                                    "quantity not traceable to any fact (>2%

# candidate place/entity tokens (skip sentence-initial capital, strip possess
words = re.findall(r"\b[A-Za-z][A-Za-z'-]+\b", sentence)
for i, w in enumerate(words):
    if i == 0:
        continue
    base = re.sub(r"'s$", "", w)          # Moon's -> Moon
    if base and base[0].isupper() and base.lower() not in STOPWORDS:
        if base.lower() in allow["strings"]:
            prov.append(f"entity:{base}")
        else:
            findings.append(Finding("surface", "entity", base,
                                    "proper noun not present in facts payload

return SentenceVerdict(sentence, ok=not findings, findings=findings, provenan

# ----- LAYER 2: semantic (int
# In production this is an LLM in its OWN context with a skeptical rubric:
# check(sentence, facts) -> (ok: bool, reason: str)
# It answers one question: "Is this sentence ENTAILED by the facts?" It must not
# context with the narration agent. The firewall treats a False verdict as fail-c
SemanticVerifier = Callable[[str, dict], tuple]

```

```

# ----- the firewall
def _split_sentences(text: str):
    return [s.strip() for s in re.split(r"(?<=[.!?])\s+", text.strip()) if s.strip()]

def firewall(flourish: str,
             facts: dict,
             semantic: Optional[SemanticVerifier] = None) -> FirewallResult:
    allow = build_allowlist(facts)
    verdicts = []
    for sent in _split_sentences(flourish):
        v = surface_scan(sent, allow)
        if v.ok and semantic is not None: # only ask the LLM if a
            ok, reason = semantic(sent, facts)
            if not ok:
                v.ok = False
                v.findings.append(Finding("semantic", "relation", sent, reason))
        verdicts.append(v)

    kept = [v.sentence for v in verdicts if v.ok]
    emitted = " ".join(kept)
    all_clean = all(v.ok for v in verdicts) and len(verdicts) > 0
    return FirewallResult(
        passed=all_clean,
        emitted_flourish=emitted,
        fell_back_to_headline=(len(kept) == 0),
        verdicts=verdicts,
    )

if __name__ == "__main__":
    facts = {
        "birth_place": "London, United Kingdom",
        "totality_over_birthplace": {
            "ever": True,
            "previous": {"date": "1715-05-03"},
            "next": {"date": "2090-09-23"},
            "drought_years": 375,
        },
        "closest_approach": {"date": "1927-06-29", "distance_km": 212, "inside_pa"},
        "saros_series": {"saros_number": 145},
    }

    demo = ("The Moon's shadow last fell on London in 1715, and will not return u
        - a gap of 375 years. In 1927 it came within about 210 km. "
        "Your eclipse belongs to Saros 145.")

```

```
r = firewall(demo, facts)
print("emitted:", r.emitted_flourish)
print("passed:", r.passed, "| findings:", r.blocked)
```